

Algoritmos de Búsqueda II

Implementaciones nucleares por algoritmo (Python y Java)

Apuntes — Grado en Inteligencia Artificial (URJC)

Curso 2025/2026

Índice

1	Bloque 2 — Trayectoriales	1
1.1	1. GRASP	1
1.2	2. Iterated Greedy	3
1.3	3. Búsqueda Tabú (Tabu Search)	4
1.4	4. Simulated Annealing	5
1.5	5. VNS — Variable Neighborhood Search	6
1.6	6. Scatter Search	8
2	Bloque 3 — Inteligencia de enjambres	9
2.1	7. PSO — Particle Swarm Optimization	9
2.2	8. ACO — Ant Colony Optimization	10
3	Bloque 4 — Evolutivos	11
3.1	9. Esquema general de un AE	11
3.2	10. Operadores de selección	12
3.3	11. Operadores de cruce	14
3.4	12. Operadores de mutación	16
3.5	13. Reemplazo (selección de supervivientes)	17
3.6	14. Algoritmos Meméticos	18
3.7	15. Programación Genética (GP)	19
3.8	16. Evolución Gramatical (GE)	21
3.9	Mapa de cierre	22

Objetivo del documento. Para cada algoritmo del temario se recoge **solo su lógica diferenciadora**: el operador, bucle o estructura que lo distingue del resto (no el algoritmo completo aplicado a una instancia concreta). Cada bloque ofrece la implementación en **Python** y en **Java**.

Convenio. Salvo que se indique lo contrario, se asume **problema de minimización** (menor *fitness* = mejor) y que un individuo/solución se evalúa con una función $f(\dots)$. Cuando un algoritmo cambia este convenio (p. ej. GRASP del examen, que maximiza), se señala explícitamente.

1 Bloque 2 — Trayectoriales

1.1 1. GRASP

Qué lo diferencia. Es un **constructivo multiarranque**. Su núcleo es la **fase constructiva voraz-aleatoria**: en cada paso no elige el mejor candidato (puro voraz) ni uno cualquiera

(puro aleatorio), sino que filtra una **Lista Restringida de Candidatos (RCL)** con los mejores y elige **uno al azar** de ella. Tras construir, aplica búsqueda local. Lo que hay que dominar es la **construcción de la RCL**.

Existen dos formas de construir la RCL: - **Por umbral de calidad** (α): $\text{umbral} = g_{\max} - \alpha \cdot (g_{\max} - g_{\min})$; entran los candidatos por encima del umbral. $\alpha=0 \rightarrow$ totalmente voraz; $\alpha=1 \rightarrow$ totalmente aleatorio. - **Por porcentaje/corte**: se quedan el $x\%$ de los mejores (variante del examen: 25 % de los mejores, garantizando ≥ 1 elemento).

Atención: En el examen (Pregunta 3) la función objetivo **maximiza** la suma, por lo que “mejor” = mayor valor. Por eso se ordena de mayor a menor.

1.1.1 Python

```
import random

# --- RCL por umbral de calidad  $\alpha()$  --- [problema de minimización: mejor = menor]
def construir_rcl_umbral(cl, alpha):
    if not cl:
        return set()
    g_min = min(cl)          # mejor coste
    g_max = max(cl)          # peor coste
    umbral = g_min + alpha * (g_max - g_min)
    return {c for c in cl if c <= umbral}

# --- RCL por porcentaje (variante del examen) --- [maximización: mejor = mayor]
def construir_rcl_porcentaje(cl, prob=0.25):
    if not cl:
        return set()
    n = max(1, round(prob * len(cl)))          # al menos 1 elemento
    mejores = sorted(cl, reverse=True)[:n]      # los 'n' mayores
    return set(mejores)

# --- Fase constructiva voraz-aleatoria (núcleo de GRASP) ---
def fase_constructiva(cl, alpha):
    solucion, candidatos = [], set(cl)
    while candidatos:
        rcl = construir_rcl_umbral(candidatos, alpha)
        elegido = random.choice(list(rcl))      # elección ALEATORIA dentro de la RCL
        solucion.append(elegido)
        candidatos.remove(elegido)              # actualización adaptativa de candidatos
    return solucion

# --- Esqueleto multiarranque ---
def grasp(cl, alpha, max_iter, busqueda_local, f):
    mejor, f_mejor = None, float("inf")
    for _ in range(max_iter):
        x = busqueda_local(fase_constructiva(cl, alpha))
        if f(x) < f_mejor:
            mejor, f_mejor = x, f(x)
    return mejor
```

1.1.2 Java

```
// RCL por porcentaje (variante del examen: 25% mejores, mínimo 1). Maximización.
public static Set<Integer> construirRCL(Set<Integer> candidateList, double probabilidad) {
    Set<Integer> rcl = new HashSet<>();
    if (candidateList.isEmpty()) return rcl;

    int n = Math.max(1, (int) Math.round(probabilidad * candidateList.size()));
```

```

List<Integer> ordenados = new ArrayList<>(candidateList);
ordenados.sort(Collections.reverseOrder()); // mejores (mayores) primero
for (int i = 0; i < n; i++) rcl.add(ordenados.get(i));
return rcl;
}

// Fase constructiva voraz-aleatoria (núcleo de GRASP). Minimización por umbral  $\alpha$ .
public static List<Integer> faseConstructiva(Set<Integer> cl, double alpha) {
    List<Integer> solucion = new ArrayList<>();
    Set<Integer> candidatos = new HashSet<>(cl);
    Random rnd = new Random();
    while (!candidatos.isEmpty()) {
        int gMin = Collections.min(candidatos), gMax = Collections.max(candidatos);
        double umbral = gMin + alpha * (gMax - gMin);
        List<Integer> rcl = new ArrayList<>();
        for (int c : candidatos) if (c <= umbral) rcl.add(c);
        int elegido = rcl.get(rnd.nextInt(rcl.size())); // elección ALEATORIA en la RCL
        solucion.add(elegido);
        candidatos.remove(elegido); // actualización adaptativa
    }
    return solucion;
}

```

1.2 2. Iterated Greedy

Qué lo diferencia. A partir de una solución ya construida, alterna una fase **destruktiva** (elimina parte de los elementos de la solución) y otra **constructiva** (los vuelve a insertar de forma voraz), aceptando o no la nueva solución según un **criterio de aceptación**. El núcleo es el par **destruir / reconstruir**.

1.2.1 Python

```

import random

def destruir(solucion, k):
    """Elimina k elementos de la solución (aleatoriamente) y los devuelve aparte."""
    sol = list(solucion)
    eliminados = random.sample(sol, k)
    for e in eliminados:
        sol.remove(e)
    return sol, eliminados

def reconstruir(parcial, eliminados, f):
    """Reinserta de forma voraz: cada elemento en la posición que minimiza el coste."""
    sol = list(parcial)
    for e in eliminados:
        mejor_pos, mejor_coste = 0, float("inf")
        for pos in range(len(sol) + 1):
            cand = sol[:pos] + [e] + sol[pos:]
            if f(cand) < mejor_coste:
                mejor_pos, mejor_coste = pos, f(cand)
        sol.insert(mejor_pos, e)
    return sol

def iterated_greedy(x0, k, parada, f, criterio_aceptacion):
    mejor = x0
    while not parada():
        parcial, elim = destruir(mejor, k) # FASE DESTRUCTIVA
        x = reconstruir(parcial, elim, f) # FASE CONSTRUCTIVA
        mejor = criterio_aceptacion(x, mejor, f) # aceptar / rechazar

```

```
return mejor
```

1.2.2 Java

```
public static List<Integer> destruir(List<Integer> solucion, int k, List<Integer> eliminadosOut) {
    List<Integer> sol = new ArrayList<>(solucion);
    Random rnd = new Random();
    for (int i = 0; i < k; i++) {
        int idx = rnd.nextInt(sol.size());
        eliminadosOut.add(sol.remove(idx));    // FASE DESTRUCTIVA
    }
    return sol;
}

public static List<Integer> reconstruir(List<Integer> parcial, List<Integer> eliminados,
    Function<List<Integer>, Double> f) {
    List<Integer> sol = new ArrayList<>(parcial);
    for (int e : eliminados) {    // FASE CONSTRUCTIVA voraz
        int mejorPos = 0; double mejorCoste = Double.POSITIVE_INFINITY;
        for (int pos = 0; pos <= sol.size(); pos++) {
            List<Integer> cand = new ArrayList<>(sol);
            cand.add(pos, e);
            double c = f.apply(cand);
            if (c < mejorCoste) { mejorCoste = c; mejorPos = pos; }
        }
        sol.add(mejorPos, e);
    }
    return sol;
}
```

1.3 3. Búsqueda Tabú (Tabu Search)

Qué lo diferencia. Es una búsqueda local que **permite empeorar** y usa **memoria adaptativa** (lista tabú a corto plazo) para no repetir movimientos recientes y escapar de ciclos. La excepción es el **criterio de aspiración**: un movimiento tabú se admite si produce la mejor solución global vista. El núcleo es: explorar un vecindario restringido $N^*(x)$, elegir el mejor admisible (aunque empeore) y **actualizar la lista tabú con su permanencia (tenure)**.

1.3.1 Python

```
def tabu_search(x0, vecinos, f, tenure, max_iter):
    x = x0
    mejor, f_mejor = x0, f(x0)
    tabu = {} # movimiento -> iteración en la que deja de ser tabú

    for it in range(max_iter):
        mejor_cand, mejor_mov, mejor_fc = None, None, float("inf")

        for cand, mov in vecinos(x):    # explora el vecindario N(x)
            es_tabu = tabu.get(mov, 0) > it
            aspira = f(cand) < f_mejor    # criterio de aspiración
            if (not es_tabu or aspira) and f(cand) < mejor_fc:
                mejor_cand, mejor_mov, mejor_fc = cand, mov, f(cand)

        if mejor_cand is None:    # aspiración por defecto: el menos tabú
            mejor_cand, mejor_mov = min(vecinos(x), key=lambda cm: tabu.get(cm[1], 0))
```

```

x = mejor_cand                                # se acepta aunque empeore
tabu[mejor_mov] = it + tenure                  # registra movimiento con su permanencia
if f(x) < f_mejor:
    mejor, f_mejor = x, f(x)
return mejor

```

1.3.2 Java

```

public static int[] tabuSearch(int[] x0, NeighbourFn vecinos, CostFn f,
                               int tenure, int maxIter) {
    int[] x = x0.clone(), mejor = x0.clone();
    double fMejor = f.apply(x0);
    Map<Integer, Integer> tabu = new HashMap<>(); // movimiento -> iteración fin tabú

    for (int it = 0; it < maxIter; it++) {
        int[] mejorCand = null, mejorMovCand = null;
        double mejorFc = Double.POSITIVE_INFINITY;

        for (Move m : vecinos.apply(x)) { // explora N(x)
            boolean esTabu = tabu.containsKey(m.id) && tabu.get(m.id) > it;
            boolean aspira = f.apply(m.sol) < fMejor; // criterio de aspiración
            if (!esTabu || aspira) && f.apply(m.sol) < mejorFc {
                mejorFc = f.apply(m.sol);
                mejorCand = m.sol; mejorMovCand = new int[]{m.id};
            }
        }
        x = mejorCand; // acepta aunque empeore
        tabu.put(mejorMovCand[0], it + tenure); // permanencia tabú (tenure)
        if (f.apply(x) < fMejor) { fMejor = f.apply(x); mejor = x.clone(); }
    }
    return mejor;
}

```

1.4 4. Simulated Annealing

Qué lo diferencia. Búsqueda local que escapa de óptimos locales aceptando empeoramientos con **probabilidad** $P = e^{(-\delta/t)}$, decreciente conforme baja la **temperatura**. El núcleo son dos cosas: el **criterio de aceptación de Metropolis** y el **esquema de enfriamiento**.

1.4.1 Python

```

import math, random

def simulated_annealing(x0, vecino_aleatorio, f, t0, alpha, nrep, t_final):
    x, fx = x0, f(x0)
    t = t0
    while t > t_final:
        for _ in range(nrep):
            cand = vecino_aleatorio(x)
            delta = f(cand) - fx
            if delta < 0: # mejora -> se acepta siempre
                x, fx = cand, f(cand)
            elif random.random() < math.exp(-delta / t): # empeora -> probabilístico
                x, fx = cand, f(cand)
        t = alpha * t # enfriamiento geométrico: t = α·t
    return x

```

```
# Otros esquemas de enfriamiento usuales:
def enfriar_boltzmann(t0, i): return t0 / (1 + math.log(1 + i)) # criterio de Boltzmann
def enfriar_cauchy(t0, i): return t0 / (1 + i) # esquema de Cauchy
def enfriar_lundy(ti, beta): return ti / (1 + beta * ti) # Lundy & Mees
```

1.4.2 Java

```
public static int[] simulatedAnnealing(int[] x0, NeighbourFn vecinoAleatorio, CostFn f,
                                     double t0, double alpha, int nrep, double tFinal) {
    int[] x = x0.clone(); double fx = f.apply(x0);
    Random rnd = new Random();
    double t = t0;
    while (t > tFinal) {
        for (int i = 0; i < nrep; i++) {
            int[] cand = vecinoAleatorio.apply(x);
            double delta = f.apply(cand) - fx;
            if (delta < 0) { // mejora -> aceptar
                x = cand; fx = f.apply(cand);
            } else if (rnd.nextDouble() < Math.exp(-delta / t)) { // empeora -> probabilístico
                x = cand; fx = f.apply(cand);
            }
        }
        t = alpha * t; // enfriamiento geométrico
    }
    return x;
}
```

1.5 5. VNS — Variable Neighborhood Search

Qué lo diferencia. Cambia **sistemáticamente la estructura de vecindad** para escapar de óptimos locales (un óptimo local en N_k no lo es en N_{k+1}). Sus dos piezas comunes son: - Shake(x, k): perturbación **aleatoria** dentro del vecindario k (diversificación). - NeighbourhoodChange(x, x', k): si x' mejora, acepta y resetea $k=1$; si no, pasa a $k+1$.

Las variantes se diferencian por **qué hacen entre el Shake y el cambio de vecindad**:

Variante	Shake (aleatorio)	Mejora
RVNS	Sí	— (sin búsqueda local)
VND	No (determinista)	Búsqueda local exhaustiva por cada vecindario
BVNS	Sí	Búsqueda local simple sobre la solución perturbada
GVNS	Sí	Un VND completo como mejora

1.5.1 Python

```
def neighbourhood_change(x, xp, k, f):
    if f(xp) < f(x):
        return xp, 1 # mejora: acepta y resetea al primer vecindario
    return x, k + 1 # no mejora: siguiente vecindario

# --- RVNS: solo Shake, sin búsqueda local ---
def rvns(x, shake, f, k_max, max_iter):
    for _ in range(max_iter):
```

```

    k = 1
    while k <= k_max:
        xp = shake(x, k)
        x, k = neighbourhood_change(x, xp, k, f)
    return x

# --- VND: determinista, mejor vecino exhaustivo por vecindario ---
def vnd(x, mejor_vecino, f, k_max):
    mejora = True
    while mejora:
        prev = x
        k = 1
        while k <= k_max:
            xp = mejor_vecino(x, k)          # exhaustivo en el vecindario k
            x, k = neighbourhood_change(x, xp, k, f)
        mejora = f(x) < f(prev)
    return x

# --- BVNS: Shake + búsqueda local simple ---
def bvns(x, shake, busqueda_local, f, k_max, max_iter):
    for _ in range(max_iter):
        k = 1
        while k <= k_max:
            xp = shake(x, k)
            xpp = busqueda_local(xp)
            x, k = neighbourhood_change(x, xpp, k, f)
    return x

# --- GVNS: Shake + VND como mejora ---
def gvns(x, shake, mejor_vecino, f, k_max, k_max_vnd, max_iter):
    for _ in range(max_iter):
        k = 1
        while k <= k_max:
            xp = shake(x, k)
            xpp = vnd(xp, mejor_vecino, f, k_max_vnd) # la mejora ES un VND completo
            x, k = neighbourhood_change(x, xpp, k, f)
    return x

```

1.5.2 Java

```

// Procedimiento común a todas las variantes.
static int[] neighbourhoodChange(int[] x, int[] xp, int[] kRef, CostFn f) {
    if (f.apply(xp) < f.apply(x)) { kRef[0] = 1; return xp; } // mejora: reset k=1
    kRef[0]++; return x;                                     // no mejora: k+1
}

// BVNS (Shake + búsqueda local). RVNS = igual sin la búsqueda local;
// GVNS = sustituir 'busquedaLocal.apply(xp)' por 'vnd(xp, ...)'.
public static int[] bvns(int[] x, ShakeFn shake, LocalSearchFn busquedaLocal,
    CostFn f, int kMax, int maxIter) {
    for (int it = 0; it < maxIter; it++) {
        int[] k = {1};
        while (k[0] <= kMax) {
            int[] xp = shake.apply(x, k[0]);
            int[] xpp = busquedaLocal.apply(xp);
            x = neighbourhoodChange(x, xpp, k, f);
        }
    }
    return x;
}

// VND determinista: el mejor vecino exhaustivo por cada vecindario.

```

```

public static int[] vnd(int[] x, BestNeighbourFn mejorVecino, CostFn f, int kMax) {
    boolean mejora = true;
    while (mejora) {
        int[] prev = x.clone(); int[] k = {1};
        while (k[0] <= kMax) x = neighbourhoodChange(x, mejorVecino.apply(x, k[0]), k, f);
        mejora = f.apply(x) < f.apply(prev);
    }
    return x;
}

```

1.6 6. Scatter Search

Qué lo diferencia. Es poblacional pero, **a diferencia de los genéticos, combina de forma sistemática (no aleatoria)** un conjunto **pequeño** de soluciones de élite, el **RefSet** (≈ 10). Su núcleo son: la **construcción del RefSet** ($b/2$ mejores + $b/2$ más diversas según una función distancia) y la **combinación** de pares.

1.6.1 Python

```

def construir_refset(P, b, f, distancia):
    """RefSet = b/2 mejores por calidad + b/2 más diversas (maximizan distancia mínima)."""
    P = sorted(P, key=f) # de mejor a peor
    refset = P[: b // 2] # mitad por CALIDAD
    restantes = P[b // 2:]
    while len(refset) < b and restantes: # mitad por DIVERSIDAD
        # elige el que maximiza su distancia mínima al RefSet actual
        elegido = max(restantes,
                       key=lambda s: min(distancia(s, r) for r in refset))
        refset.append(elegido)
        restantes.remove(elegido)
    return refset

def combinar(s1, s2):
    """Combinación sistemática (p. ej. media/centroide componente a componente)."""
    return [(a + b) / 2 for a, b in zip(s1, s2)]

def actualizar_refset(refset, x, f):
    """Inserta x si mejora a la peor del RefSet (actualización por calidad)."""
    peor = max(refset, key=f)
    if f(x) < f(peor):
        refset.remove(peor)
        refset.append(x)
    return refset

def scatter_search(P, b, f, distancia, mejora):
    refset = construir_refset(P, b, f, distancia)
    nueva = True
    while nueva:
        nueva = False
        for i in range(len(refset)): # todos los pares del RefSet
            for j in range(i + 1, len(refset)):
                x = mejora(combinar(refset[i], refset[j]))
                if f(x) < f(max(refset, key=f)):
                    refset = actualizar_refset(refset, x, f)
                    nueva = True
    return min(refset, key=f)

```

1.6.2 Java


```
// Construcción del RefSet: b/2 por calidad + b/2 por diversidad.
public static List<double[]> construirRefSet(List<double[]> P, int b,
                                           CostFnD f, DistFn dist) {
    P.sort(Comparator.comparingDouble(f::apply));
    List<double[]> refset = new ArrayList<>(P.subList(0, b / 2)); // CALIDAD
    List<double[]> restantes = new ArrayList<>(P.subList(b / 2, P.size()));
    while (refset.size() < b && !restantes.isEmpty()) { // DIVERSIDAD
        double[] elegido = null; double mejorMinDist = -1;
        for (double[] s : restantes) {
            double minDist = Double.POSITIVE_INFINITY;
            for (double[] r : refset) minDist = Math.min(minDist, dist.apply(s, r));
            if (minDist > mejorMinDist) { mejorMinDist = minDist; elegido = s; }
        }
        refset.add(elegido); restantes.remove(elegido);
    }
    return refset;
}

// Combinación sistemática (centroide componente a componente).
public static double[] combinar(double[] s1, double[] s2) {
    double[] z = new double[s1.length];
    for (int i = 0; i < s1.length; i++) z[i] = (s1[i] + s2[i]) / 2.0;
    return z;
}
```

2 Bloque 3 — Inteligencia de enjambres

2.1 7. PSO — Particle Swarm Optimization

Qué lo diferencia. Cada partícula tiene **posición** y **velocidad** (vectores). La velocidad se actualiza combinando tres términos: **inercia** (w), **componente cognitiva** (atracción a su mejor histórico $pBest$) y **componente social** (atracción al mejor del enjambre $gBest$). El núcleo es la **ecuación de actualización de velocidad**:

$$V(t+1) = w \cdot V(t) + c1 \cdot r1 \cdot (pBest - X(t)) + c2 \cdot r2 \cdot (gBest - X(t))$$

2.1.1 Python

```
import random

def actualizar_particula(x, v, p_best, g_best, w, c1, c2):
    nueva_v, nueva_x = [], []
    for d in range(len(x)):
        r1, r2 = random.random(), random.random()
        vd = (w * v[d]
              + c1 * r1 * (p_best[d] - x[d])
              + c2 * r2 * (g_best[d] - x[d]))
        nueva_v.append(vd)
        nueva_x.append(x[d] + vd)
    return nueva_x, nueva_v

def pso(crear_enjambre, f, w, c1, c2, n, max_iter):
    X = crear_enjambre(n)
    V = [[0.0] * len(x) for x in X]
    p_best = [list(x) for x in X]
    g_best = min(X, key=f)
    for _ in range(max_iter):
        for i in range(n):
            X[i], V[i] = actualizar_particula(X[i], V[i], p_best[i], g_best, w, c1, c2)
```

```

        if f(X[i]) < f(p_best[i]):
            p_best[i] = list(X[i])          # mejor histórico de la partícula
        if f(X[i]) < f(g_best):
            g_best = list(X[i])             # mejor global del enjambre
    return g_best

```

2.1.2 Java

```

public static double[][] actualizarParticula(double[] x, double[] v,
    double[] pBest, double[] gBest, double w, double c1, double c2) {
    double[] nuevaV = new double[x.length], nuevaX = new double[x.length];
    Random rnd = new Random();
    for (int d = 0; d < x.length; d++) {
        double r1 = rnd.nextDouble(), r2 = rnd.nextDouble();
        nuevaV[d] = w * v[d]                // inercia
            + c1 * r1 * (pBest[d] - x[d])    // cognitiva
            + c2 * r2 * (gBest[d] - x[d]);    // social
        nuevaX[d] = x[d] + nuevaV[d];        // X(t+1) = X(t) + V(t+1)
    }
    return new double[][]{nuevaX, nuevaV};
}

```

2.2 8. ACO — Ant Colony Optimization

Qué lo diferencia. Construye soluciones recorriendo un grafo; la decisión de cada arista es **probabilística** según **feromona** (τ , experiencia) y **visibilidad** ($\eta = 1/d$, heurística). Tras completar todos los recorridos, **se evaporan y refuerzan las feromonas**. Núcleo: **probabilidad de transición, selección por ruleta y actualización de feromonas**.

$$p_{ij}^k = (\tau_{ij})^\alpha (\eta_{ij})^\beta / \sum_l (\tau_{il})^\alpha (\eta_{il})^\beta$$

2.2.1 Python

```

import random

def probabilidad_transicion(i, disponibles, tau, eta, alpha, beta):
    pesos = [(tau[i][j] ** alpha) * (eta[i][j] ** beta) for j in disponibles]
    total = sum(pesos)
    return [w / total for w in pesos]          # probabilidades normalizadas

def ruleta(probabilidades):
    r, acumulado = random.random(), 0.0
    for i, p in enumerate(probabilidades):
        acumulado += p
        if r <= acumulado:
            return i                          # índice seleccionado
    return len(probabilidades) - 1

def actualizar_feromonas(tau, recorridos, longitudes, rho, Q):
    n = len(tau)
    for i in range(n):
        for j in range(n):
            tau[i][j] *= rho                  # EVAPORACIÓN:  $\tau = \rho \cdot \tau$   $\rho(<1)$ 
    for k, camino in enumerate(recorridos):
        aporte = Q / longitudes[k]            #  $\Delta\tau = Q / L_k$  (mejor solución -> más feromona)
        for (i, j) in zip(camino, camino[1:]):
            tau[i][j] += aporte                # REFUERZO
    return tau

```

```
def construir_hormiga(tau, eta, alpha, beta, n_nodos):
    actual = random.randrange(n_nodos)
    camino, visitados = [actual], {actual}
    while len(visitados) < n_nodos:
        disponibles = [j for j in range(n_nodos) if j not in visitados] # sin ciclos
        probs = probabilidad_transicion(actual, disponibles, tau, eta, alpha, beta)
        actual = disponibles[ruleta(probs)]
        camino.append(actual); visitados.add(actual)
    return camino
```

2.2.2 Java

```
public static double[] probabilidadTransicion(int i, List<Integer> disponibles,
    double[][] tau, double[][] eta, double alpha, double beta) {
    double[] pesos = new double[disponibles.size()]; double total = 0;
    for (int idx = 0; idx < disponibles.size(); idx++) {
        int j = disponibles.get(idx);
        pesos[idx] = Math.pow(tau[i][j], alpha) * Math.pow(eta[i][j], beta);
        total += pesos[idx];
    }
    for (int idx = 0; idx < pesos.length; idx++) pesos[idx] /= total;
    return pesos;
}

public static int ruleta(double[] probabilidades) {
    double r = new Random().nextDouble(), acumulado = 0;
    for (int i = 0; i < probabilidades.length; i++) {
        acumulado += probabilidades[i];
        if (r <= acumulado) return i;
    }
    return probabilidades.length - 1;
}

public static void actualizarFeromonas(double[][] tau, List<int[]> recorridos,
    double[] longitudes, double rho, double Q) {
    for (int i = 0; i < tau.length; i++)
        for (int j = 0; j < tau.length; j++) tau[i][j] *= rho; // EVAPORACIÓN
    for (int k = 0; k < recorridos.size(); k++) {
        int[] camino = recorridos.get(k);
        double aporte = Q / longitudes[k]; //  $\Delta\tau = Q / L_k$ 
        for (int idx = 0; idx < camino.length - 1; idx++)
            tau[camino[idx]][camino[idx + 1]] += aporte; // REFUERZO
    }
}
```

3 Bloque 4 — Evolutivos

3.1 9. Esquema general de un AE

Qué lo diferencia. Es el **bucle base** que comparten todos los evolutivos: inicializar y evaluar la población, y repetir *selección* → *recombinación* → *mutación* → *evaluación* → *reemplazo* hasta cumplir una condición de parada. Las secciones 10–13 detallan cada operador.

3.1.1 Python

```
def algoritmo_evolutivo(init_poblacion, f, seleccion, cruce, mutacion,
                        reemplazo, parada):
    poblacion = init_poblacion()
    fitness = [f(ind) for ind in poblacion]
    while not parada():
        padres = seleccion(poblacion, fitness)      # 1. selección
        hijos = cruce(padres)                       # 2. recombinación
        hijos = [mutacion(h) for h in hijos]         # 3. mutación
        f_hijos = [f(h) for h in hijos]             # 4. evaluación
        poblacion, fitness = reemplazo(poblacion, fitness, hijos, f_hijos) # 5. reemplazo
    return min(poblacion, key=f)
```

3.1.2 Java

```
public static int[] algoritmoEvolutivo(Supplier<List<int[]>> initPob, CostFn f,
    SeleccionFn seleccion, CruceFn cruce, MutacionFn mutacion,
    ReemplazoFn reemplazo, Supplier<Boolean> parada) {
    List<int[]> poblacion = initPob.get();
    while (!parada.get()) {
        List<int[]> padres = seleccion.apply(poblacion, f); // 1. selección
        List<int[]> hijos = cruce.apply(padres);           // 2. recombinación
        hijos.replaceAll(mutacion::apply);                 // 3. mutación (+4 eval interna)
        poblacion = reemplazo.apply(poblacion, hijos, f);  // 5. reemplazo
    }
    return poblacion.stream().min(Comparator.comparingDouble(f::apply)).get();
}
```

3.2 10. Operadores de selección

Qué los diferencia. Trabajan sobre individuos completos (independientes de la representación). Se distinguen por **cómo asignan la presión selectiva**: - **Ruleta (FPS)**: probabilidad proporcional al fitness; sensible a transposición y a convergencia prematura. - **SUS**: una sola “rueda” con n brazos equiespaciados → reparto más justo. - **Ranking**: probabilidad según el **rango** (orden), no el valor absoluto. - **Torneo**: elige el mejor de k aleatorios; no usa estadísticas globales (ideal para paralelizar).

Nota: en problemas de **minimización** conviene transformar el fitness (p. ej. usar $1/(1+f)$ o $\text{máx}-f$) para que “mejor” reciba mayor probabilidad. Abajo se asume maximización en ruleta/SUS por claridad.

3.2.1 Python

```
import random

def seleccion_ruleta(poblacion, fitness):
    total = sum(fitness)
    r, acum = random.uniform(0, total), 0
    for ind, fit in zip(poblacion, fitness):
        acum += fit
        if acum >= r:
            return ind # un giro = un seleccionado

def seleccion_sus(poblacion, fitness, n):
    total = sum(fitness)
    paso = total / n
    inicio = random.uniform(0, paso)
```

```

punteros = [inicio + i * paso for i in range(n)] # n brazos equiespaciados
seleccionados, acum, i = [], 0, 0
fa = list(zip(poblacion, fitness))
for p in punteros:
    while acum < p:
        acum += fa[i][1]; i += 1
    seleccionados.append(fa[i - 1][0])
return seleccionados

def seleccion_ranking(poblacion, fitness):
    # rango 1 (peor) .. mu (mejor); probabilidad proporcional al rango
    orden = sorted(range(len(poblacion)), key=lambda k: fitness[k]) # peor->mejor (min)
    rangos = [0] * len(poblacion)
    for r, idx in enumerate(orden, start=1):
        rangos[idx] = r
    total = sum(rangos)
    rr, acum = random.uniform(0, total), 0
    for ind, rg in zip(poblacion, rangos):
        acum += rg
        if acum >= rr:
            return ind

def seleccion_torneo(poblacion, fitness, k):
    aspirantes = random.sample(range(len(poblacion)), k)
    ganador = min(aspirantes, key=lambda idx: fitness[idx]) # mejor = menor (min)
    return poblacion[ganador]

```

3.2.2 Java

```

public static int[] seleccionRuleta(List<int[]> pob, double[] fit) {
    double total = Arrays.stream(fit).sum();
    double r = new Random().nextDouble() * total, acum = 0;
    for (int i = 0; i < pob.size(); i++) {
        acum += fit[i];
        if (acum >= r) return pob.get(i);
    }
    return pob.get(pob.size() - 1);
}

public static int[] seleccionTorneo(List<int[]> pob, double[] fit, int k) {
    Random rnd = new Random();
    int mejor = rnd.nextInt(pob.size());
    for (int i = 1; i < k; i++) {
        int c = rnd.nextInt(pob.size());
        if (fit[c] < fit[mejor]) mejor = c; // mejor = menor (minimización)
    }
    return pob.get(mejor);
}

// Ranking: probabilidad proporcional al rango (l=peor .. mu=mejor).
public static int[] seleccionRanking(List<int[]> pob, double[] fit) {
    Integer[] orden = new Integer[pob.size()];
    for (int i = 0; i < orden.length; i++) orden[i] = i;
    Arrays.sort(orden, (a, b) -> Double.compare(fit[b], fit[a])); // mejor(menor) último -> rango alto
    int[] rango = new int[pob.size()];
    for (int r = 0; r < orden.length; r++) rango[orden[r]] = r + 1;
    int total = pob.size() * (pob.size() + 1) / 2;
    double rr = new Random().nextDouble() * total; int acum = 0;
    for (int i = 0; i < pob.size(); i++) {
        acum += rango[i];
        if (acum >= rr) return pob.get(i);
    }
}

```

```

    return pob.get(pob.size() - 1);
}

```

3.3 11. Operadores de cruce

Qué los diferencia. Mezclan información de dos progenitores. La elección depende de la **representación**: - **Binaria/entera**: 1-punto, N-puntos (sesgo posicional), uniforme (sin sesgo posicional). - **Real**: discreto (cada gen de un padre) y aritmético ($z = \alpha \cdot x + (1-\alpha) \cdot y \rightarrow$). - **Permutaciones**: OX1 (preserva orden relativo), PMX (posición + adyacencia), CX (alelo+posición por ciclos). Los cruces normales romperían la condición de permutación.

3.3.1 Python

```

import random

# --- Binaria/entera ---
def cruce_un_punto(p1, p2):
    c = random.randint(1, len(p1) - 1)
    return p1[:c] + p2[c:], p2[:c] + p1[c:]

def cruce_n_puntos(p1, p2, n):
    puntos = sorted(random.sample(range(1, len(p1)), n))
    h1, h2, intercambia = [], [], False
    prev = 0
    for p in puntos + [len(p1)]:
        if intercambia: h1 += p2[prev:p]; h2 += p1[prev:p]
        else:           h1 += p1[prev:p]; h2 += p2[prev:p]
        intercambia = not intercambia; prev = p
    return h1, h2

def cruce_uniforme(p1, p2):
    h1, h2 = [], []
    for a, b in zip(p1, p2):
        if random.random() < 0.5: h1.append(a); h2.append(b)
        else:                     h1.append(b); h2.append(a)
    return h1, h2

# --- Real: cruce aritmético (intermedio) ---
def cruce_aritmetico(p1, p2, alpha):
    h1 = [alpha * a + (1 - alpha) * b for a, b in zip(p1, p2)]
    h2 = [alpha * b + (1 - alpha) * a for a, b in zip(p1, p2)]
    return h1, h2

# --- Permutaciones: Order Crossover (OX1) ---
def cruce_ox1(p1, p2):
    n = len(p1)
    a, b = sorted(random.sample(range(n), 2))
    hijo = [None] * n
    hijo[a:b] = p1[a:b] # 1. copia un tramo de P1
    resto = [g for g in p2[b:] + p2[:b] if g not in hijo] # 2. resto en orden de P2
    pos = b
    for g in resto:
        if pos >= n: pos = 0
        while hijo[pos] is not None:
            pos += 1
        if pos >= n: pos = 0
        hijo[pos] = g; pos += 1
    return hijo

# --- Permutaciones: Cycle Crossover (CX) ---

```

```

def cruce_cx(p1, p2):
    n = len(p1)
    hijo = [None] * n
    idx, usar_p1 = 0, True
    visitados = set()
    while len(visitados) < n:
        if idx in visitados:                # busca el siguiente ciclo
            idx = next(i for i in range(n) if i not in visitados)
            usar_p1 = not usar_p1
        ciclo, j = [], idx
        while j not in visitados:           # 1. identifica un ciclo
            visitados.add(j); ciclo.append(j)
            j = p1.index(p2[j])
        fuente = p1 if usar_p1 else p2      # 2. copia alternando padre
        for k in ciclo: hijo[k] = fuente[k]
        usar_p1 = not usar_p1
    return hijo

```

3.3.2 Java

```

public static int[][] cruceUnPunto(int[] p1, int[] p2) {
    int c = 1 + new Random().nextInt(p1.length - 1);
    int[] h1 = new int[p1.length], h2 = new int[p1.length];
    for (int i = 0; i < p1.length; i++) {
        h1[i] = (i < c) ? p1[i] : p2[i];
        h2[i] = (i < c) ? p2[i] : p1[i];
    }
    return new int[][]{h1, h2};
}

public static int[][] cruceUniforme(int[] p1, int[] p2) {
    Random rnd = new Random();
    int[] h1 = new int[p1.length], h2 = new int[p1.length];
    for (int i = 0; i < p1.length; i++) {
        boolean cara = rnd.nextBoolean();
        h1[i] = cara ? p1[i] : p2[i];
        h2[i] = cara ? p2[i] : p1[i];        // copia inversa
    }
    return new int[][]{h1, h2};
}

public static double[] cruceAritmetico(double[] p1, double[] p2, double alpha) {
    double[] h = new double[p1.length];
    for (int i = 0; i < p1.length; i++) h[i] = alpha * p1[i] + (1 - alpha) * p2[i];
    return h;
}

// Order Crossover (OX1) para permutaciones.
public static int[] cruceOX1(int[] p1, int[] p2) {
    int n = p1.length; Random rnd = new Random();
    int a = rnd.nextInt(n), b = rnd.nextInt(n);
    if (a > b) { int t = a; a = b; b = t; }
    int[] hijo = new int[n]; Arrays.fill(hijo, -1);
    Set<Integer> usados = new HashSet<>();
    for (int i = a; i < b; i++) { hijo[i] = p1[i]; usados.add(p1[i]); }    // tramo de P1
    int pos = b;
    for (int k = 0; k < n; k++) {
        int g = p2[(b + k) % n];
        if (!usados.contains(g)) {
            if (pos >= n) pos = 0;
            hijo[pos++] = g; usados.add(g);
        }
    }
}

```

```

    }
    return hijo;
}

```

3.4 12. Operadores de mutación

Qué los diferencia. Introducen **nuevos alelos** (a diferencia del cruce). Dependen de la representación: - **Binaria:** *bit-flip* independiente por gen con probabilidad p_m . - **Entera:** *creep* (cambio a un valor cercano) o elección aleatoria (categóricas). - **Real:** uniforme o gaussiana $N(0, \sigma)$. - **Permutaciones:** deben cambiar ≥ 2 valores para no romper la permutación $\rightarrow$ inserción, intercambio (swap), inversión, mezcla (scramble). Aquí p_m es la probabilidad de aplicar el operador **una vez a toda la cadena**.

3.4.1 Python

```

import random

def mut_bitflip(crom, pm):
    # binaria: gen a gen
    return [1 - g if random.random() < pm else g for g in crom]

def mut_creep(crom, pm, paso, lb, ub):
    # entera: pequeño desplazamiento
    out = []
    for g in crom:
        if random.random() < pm:
            g = max(lb, min(ub, g + random.choice([-paso, paso])))
        out.append(g)
    return out

def mut_gaussiana(crom, pm, sigma, lb, ub):
    # real: N(0, sigma)
    out = []
    for g in crom:
        if random.random() < pm:
            g = max(lb, min(ub, g + random.gauss(0, sigma)))
        out.append(g)
    return out

# --- Permutaciones ---
def mut_swap(perm):
    # intercambio de 2 posiciones
    p = list(perm); i, j = random.sample(range(len(p)), 2)
    p[i], p[j] = p[j], p[i]; return p

def mut_insercion(perm):
    # mueve un elemento junto a otro
    p = list(perm); i, j = sorted(random.sample(range(len(p)), 2))
    valor = p.pop(j); p.insert(i + 1, valor); return p

def mut_inversion(perm):
    # invierte un subtramo
    p = list(perm); i, j = sorted(random.sample(range(len(p)), 2))
    p[i:j + 1] = reversed(p[i:j + 1]); return p

def mut_scramble(perm):
    # reorganiza aleatoriamente un subconjunto
    p = list(perm); i, j = sorted(random.sample(range(len(p)), 2))
    sub = p[i:j + 1]; random.shuffle(sub); p[i:j + 1] = sub; return p

```

3.4.2 Java

```

public static int[] mutBitflip(int[] crom, double pm) {
    Random rnd = new Random(); int[] out = crom.clone();

```



```

    for (int i = 0; i < out.length; i++)
        if (rnd.nextDouble() < pm) out[i] = 1 - out[i];
    return out;
}

public static double[] mutGaussiana(double[] crom, double pm, double sigma,
                                   double lb, double ub) {
    Random rnd = new Random(); double[] out = crom.clone();
    for (int i = 0; i < out.length; i++)
        if (rnd.nextDouble() < pm)
            out[i] = Math.max(lb, Math.min(ub, out[i] + rnd.nextGaussian() * sigma));
    return out;
}

public static int[] mutInversion(int[] perm) {           // permutaciones
    int[] p = perm.clone(); Random rnd = new Random();
    int i = rnd.nextInt(p.length), j = rnd.nextInt(p.length);
    if (i > j) { int t = i; i = j; j = t; }
    while (i < j) { int t = p[i]; p[i] = p[j]; p[j] = t; i++; j--; }
    return p;
}

public static int[] mutSwap(int[] perm) {
    int[] p = perm.clone(); Random rnd = new Random();
    int i = rnd.nextInt(p.length), j = rnd.nextInt(p.length);
    int t = p[i]; p[i] = p[j]; p[j] = t;
    return p;
}

```

3.5 13. Reemplazo (selección de supervivientes)

Qué lo diferencia. Decide qué individuos pasan a la siguiente generación: - **Generacional (GGA):** la descendencia reemplaza a toda la población (brecha generacional = 1). - **Steady-state (SSGA):** se incorporan **unos pocos** descendientes (*indivss*) reemplazando a los peores de la población. - **Elitismo:** se garantiza la supervivencia del/los mejor(es).

Reemplazo steady-state con elitismo (Pregunta 4 del examen, minimización).

La implementación correcta: tomar los **mejores** *indivss* **descendientes** y sustituir con ellos a los **peores** *indivss* de la población, solo si los mejoran. Así nunca se elimina al mejor (elitismo) y sí se insertan los descendientes (el código del examen fallaba precisamente en esto, entrando en bucle infinito sin insertar). Con *poblacion* = [20,18,16,14,12,10,8,6,4,2], *descendientes* = [19,17,15,13,11,9,7,5,3,1] e *indivss* = 2, devuelve [1,2,3,4,6,8,10,12,14,16].

3.5.1 Python

```

def reemplazo_generacional(poblacion, fitness, hijos, f_hijos, elitismo=True):
    if elitismo:
        elite = min(range(len(poblacion)), key=lambda k: fitness[k])
        peor_hijo = max(range(len(hijos)), key=lambda k: f_hijos[k])
        hijos = list(hijos); f_hijos = list(f_hijos)
        hijos[peor_hijo] = poblacion[elite]           # conserva al mejor
        f_hijos[peor_hijo] = fitness[elite]
    return hijos, f_hijos

def reemplazo_steady_state(poblacion, descendientes, indiv_ss):
    """SSGA con elitismo de 1 (minimización). Inserta los mejores indiv_ss descendientes
    sustituyendo a los peores de la población, solo si los mejoran."""
    nueva = list(poblacion)

```

```

mejores_desc = sorted(descendientes)[:indiv_ss] # menor fitness = mejor
for desc in mejores_desc:
    nueva.sort() # el peor queda al final
    if desc < nueva[-1]: # elitismo: solo toca al peor
        nueva[-1] = desc
    return sorted(nueva)

# Verificación con el caso del examen:
# reemplazo_steady_state([20,18,16,14,12,10,8,6,4,2],
#                        [19,17,15,13,11,9,7,5,3,1], 2)
# -> [1, 2, 3, 4, 6, 8, 10, 12, 14, 16]

```

3.5.2 Java

```

// Reemplazo steady-state con elitismo de 1 (minimización).
// Corrige la Pregunta 4: inserta descendientes y no entra en bucle infinito.
public static List<Integer> reemplazo(List<Integer> poblacion,
                                     List<Integer> descendientes, int indivSS) {
    List<Integer> nueva = new ArrayList<>(poblacion);
    List<Integer> desc = new ArrayList<>(descendientes);
    Collections.sort(desc); // mejores (menores) primero

    for (int i = 0; i < indivSS && i < desc.size(); i++) {
        Collections.sort(nueva); // el peor queda al final
        int peor = nueva.get(nueva.size() - 1);
        if (desc.get(i) < peor) { // elitismo: solo sustituye al peor
            nueva.set(nueva.size() - 1, desc.get(i));
        }
    }
    Collections.sort(nueva);
    return nueva;
}
// reemplazo([20,18,16,14,12,10,8,6,4,2], [19,17,15,13,11,9,7,5,3,1], 2)
// -> [1, 2, 3, 4, 6, 8, 10, 12, 14, 16]

```

3.6 14. Algoritmos Meméticos

Qué los diferencia. Son un **AE + búsqueda local** aplicada a los individuos (típicamente a los descendientes): “*global search poblacional casada con búsqueda local heurística*”. La pieza distintiva es **cómo se transmite la mejora local**: - **Lamarckiano**: el individuo se **reemplaza** por su versión mejorada (cambia el genotipo → se hereda). - **Baldwiniano**: el individuo conserva su genotipo pero adopta el **fitness** del vecino mejorado (solo cambia la evaluación; el “efecto Baldwin”).

3.6.1 Python

```

def busqueda_local(ind, vecinos, f):
    """Best improvement: devuelve el mejor vecino encontrado."""
    mejor, f_mejor = ind, f(ind)
    for v in vecinos(ind):
        if f(v) < f_mejor:
            mejor, f_mejor = v, f(v)
    return mejor, f_mejor

def aplicar_meme(ind, vecinos, f, modo="lamarck"):
    mejorado, f_mejorado = busqueda_local(ind, vecinos, f)
    if modo == "lamarck":

```

```

        return mejorado, f_mejorado          # cambia genotipo (se hereda)
    else: # baldwin
        return ind, f_mejorado              # conserva genotipo, hereda solo el fitness

def algoritmo_memetico(init_pob, vecinos, f, seleccion, cruce, mutacion,
                      reemplazo, parada, modo="lamarck"):
    poblacion = init_pob()
    while not parada():
        padres = seleccion(poblacion)
        hijos = [mutacion(h) for h in cruce(padres)]
        # núcleo memético: búsqueda local sobre cada descendiente
        evaluados = [aplicar_meme(h, vecinos, f, modo) for h in hijos]
        nuevos = [g for g, _ in evaluados]
        poblacion = reemplazo(poblacion, nuevos)
    return min(poblacion, key=f)

```

3.6.2 Java

```

// Best improvement.
static int[] busquedaLocal(int[] ind, NeighbourFn vecinos, CostFn f) {
    int[] mejor = ind.clone(); double fMejor = f.apply(ind);
    for (int[] v : vecinos.apply(ind))
        if (f.apply(v) < fMejor) { fMejor = f.apply(v); mejor = v.clone(); }
    return mejor;
}

// Lamarckiano: devuelve el genotipo mejorado (se hereda).
static int[] memelamarck(int[] ind, NeighbourFn vecinos, CostFn f) {
    return busquedaLocal(ind, vecinos, f);
}

// Baldwiniano: conserva el genotipo original; solo se usaría el fitness del vecino mejorado.
static double memeBaldwinFitness(int[] ind, NeighbourFn vecinos, CostFn f) {
    return f.apply(busquedaLocal(ind, vecinos, f)); // genotipo intacto, fitness mejorado
}

```

3.7 15. Programación Genética (GP)

Qué la diferencia. Los individuos son **árboles** (programas/expresiones), no cadenas lineales. Por tanto sus operadores son distintos: el **cruce intercambia subárboles** y la **mutación sustituye un subárbol** por otro aleatorio. La inicialización usa *full*, *grow* o *ramped half-and-half*. (Cuidado: el examen marcaba como **falso** que GP use “cromosomas lineales de longitud fija” — eso es justamente lo contrario de GP.)

3.7.1 Python

```

import random, copy

class Nodo:
    def __init__(self, valor, hijos=None):
        self.valor = valor          # función ('+', '*', 'sin'...) o terminal ('x', 3)
        self.hijos = hijos or []

FUNCIONES = {'+': 2, '-': 2, '*': 2, 'sin': 1} # función -> aridad
TERMINALES = ['x', 'y', 1, 2, 3]

def generar_arbol(prof_max, metodo='grow'):

```

```

# full: hasta prof_max solo funciones; grow: mezcla; terminal al llegar a 0
if prof_max == 0 or (metodo == 'grow' and random.random() < 0.3):
    return Nodo(random.choice(TERMINALES))
f = random.choice(list(FUNCIONES))
return Nodo(f, [generar_arbol(prof_max - 1, metodo) for _ in range(FUNCIONES[f])])

def ramped_half_and_half(n, prof_max):
    pob = []
    for i in range(n):
        metodo = 'full' if i % 2 == 0 else 'grow'
        pob.append(generar_arbol(random.randint(1, prof_max), metodo))
    return pob

def _listar_nodos(arbol, acc=None):
    acc = acc if acc is not None else []
    acc.append(arbol)
    for h in arbol.hijos: _listar_nodos(h, acc)
    return acc

def cruce_subarboles(p1, p2):
    h1, h2 = copy.deepcopy(p1), copy.deepcopy(p2)
    n1 = random.choice(_listar_nodos(h1))
    n2 = random.choice(_listar_nodos(h2))
    n1.valor, n2.hijos, n2.valor, n2.hijos = n2.valor, n2.hijos, n1.valor, n1.hijos
    return h1, h2 # se intercambian subárboles

def mutacion_subarbol(arbol, prof_max):
    h = copy.deepcopy(arbol)
    nodo = random.choice(_listar_nodos(h))
    nuevo = generar_arbol(prof_max, 'grow')
    nodo.valor, nodo.hijos = nuevo.valor, nuevo.hijos # subárbol -> árbol aleatorio
    return h

```

3.7.2 Java

```

class Nodo {
    String valor; List<Nodo> hijos = new ArrayList<>();
    Nodo(String v) { valor = v; }
}

static final Map<String,Integer> FUNCIONES = Map.of("+",2,"-",2,"*",2,"sin",1);
static final String[] TERMINALES = {"x","y","1","2","3"};
static final Random RND = new Random();

static Nodo generarArbol(int profMax, boolean grow) {
    if (profMax == 0 || (grow && RND.nextDouble() < 0.3))
        return new Nodo(TERMINALES[RND.nextInt(TERMINALES.length)]);
    List<String> fs = new ArrayList<>(FUNCIONES.keySet());
    String f = fs.get(RND.nextInt(fs.size()));
    Nodo n = new Nodo(f);
    for (int i = 0; i < FUNCIONES.get(f); i++) n.hijos.add(generarArbol(profMax - 1, grow));
    return n;
}

static void listarNodos(Nodo a, List<Nodo> acc) {
    acc.add(a);
    for (Nodo h : a.hijos) listarNodos(h, acc);
}

// Cruce: intercambia un subárbol aleatorio entre dos copias.
static void cruceSubarboles(Nodo h1, Nodo h2) {
    List<Nodo> l1 = new ArrayList<>(), l2 = new ArrayList<>();

```

```

listarNodos(h1, l1); listarNodos(h2, l2);
Nodo n1 = l1.get(RND.nextInt(l1.size())); n2 = l2.get(RND.nextInt(l2.size()));
String v = n1.valor; List<Nodo> hs = n1.hijos;
n1.valor = n2.valor; n1.hijos = n2.hijos;
n2.valor = v; n2.hijos = hs;
}

```

3.8 16. Evolución Gramatical (GE)

Qué la diferencia. Une lo mejor de AG y GP: **genotipo = vector lineal de enteros** (operadores sencillos, como en AG) y **fenotipo = programa/árbol** (expresividad, como en GP). La traducción se hace con una **gramática BNF** y la regla de mapeo producción = código $\hookrightarrow n \bmod n^{\circ}_{\text{de alternativas}}$. Si se agotan los codones, se hace **WRAP** (volver al inicio); superado el límite de wraps, el individuo es **inválido**. El núcleo es la **función de mapeo genotipo $\rightarrow$ fenotipo**.

3.8.1 Python

```

def mapear(genotipo, gramatica, simbolo_inicial, max_wraps=2):
    """genotipo: lista de enteros (codones). gramatica: {no_terminal: [alternativas]}.
    Cada alternativa es una lista de símbolos (no terminales o terminales)."""
    salida, pila = [], [simbolo_inicial]
    i, wraps = 0, 0
    while pila:
        simbolo = pila.pop(0)
        if simbolo not in gramatica:           # terminal -> a la salida
            salida.append(simbolo)
            continue
        if i >= len(genotipo):                 # se agotaron los codones
            wraps += 1
            if wraps > max_wraps:
                return None                    # individuo INVÁLIDO
            i = 0                             # WRAP: volver al inicio
        alternativas = gramatica[simbolo]
        eleccion = genotipo[i] % len(alternativas) # codón mod n° alternativas
        i += 1
        pila = list(alternativas[eleccion]) + pila # expandir el no terminal
    return "".join(salida)

# Ejemplo de gramática y mapeo (resultado esperado: 'sin(C*x)')
gramatica = {
    "<func>": [[("<expr>", "<op>", "<expr>"), ("<preop>", "(", "<expr>", ")")],
    "<expr>": [{"(", "<cte>", "<op>", "<var>", ")"}, ["<var>"]],
    "<op>": [{"+"}, [{"-"}, [{"/"}, [{"*"}],
    "<var>": [{"x"}, [{"y"}, [{"z"}],
    "<cte>": [{"A"}, [{"B"}, [{"C"}, [{"D"}, [{"E"}],
    "<preop>": [{"sin"}, [{"cos"}],
}

# mapear([15, 200, 22, 17, 123], gramatica, "<func>") -> 'sin((C*x))' (los paréntesis salen de la
    \hookrightarrow regla <expr>)

```

3.8.2 Java

```

public static String mapear(int[] genotipo, Map<String, List<List<String>>> gramatica,
    String inicial, int maxWraps) {
    StringBuilder salida = new StringBuilder();
    Deque<String> pila = new ArrayDeque<>();
}

```

```

pila.addFirst(inicial);
int i = 0, wraps = 0;
while (!pila.isEmpty()) {
    String simbolo = pila.pollFirst();
    if (!gramatica.containsKey(simbolo)) { salida.append(simbolo); continue; } // terminal
    if (i >= genotipo.length) { // codones agotados
        if (++wraps > maxWraps) return null; // INVÁLIDO
        i = 0; // WRAP
    }
    List<List<String>> alts = gramatica.get(simbolo);
    int eleccion = Math.floorMod(genotipo[i++], alts.size()); // codón mod nº alternativas
    List<String> prod = alts.get(eleccion);
    for (int k = prod.size() - 1; k >= 0; k--) pila.addFirst(prod.get(k)); // expandir
}
return salida.toString();
}
// mapear({15,200,22,17,123}, gramatica, "<func>", 2) -> "sin((C*x))"

```

3.9 Mapa de cierre

Trayectoriales	Enjambres	Evolutivos
- GRASP (RCL)	- PSO (V,X)	- AE (bucle base)
- Iterated Greedy	- ACO (tau,eta,ruleta)	- Seleccion (ruleta/SUS/ranking/torneo)
- Tabu (memoria)		- Cruce (lpt/Npt/unif/aritm/OX1/PMX/CX)
- SA (accept. $e^{-d/t}$)		- Mutacion (bitflip/creep/gauss/perm)
- VNS (shake+NC:		- Reemplazo (gen/steady-state+elitismo)
RVNS/VND/BVNS/GVNS)		- Memeticos (AE+BL: Lamarck/Baldwin)
- Scatter Search (RefSet)		- GP (arboles, subarboles)
		- GE (codon mod regla + wrap)